TABLE DES MATIÈRES

1. OUTILS 3

INTRODUCTION 3
INSTALLATION DES OUTILS 3
L'ENVIRONNEMENT DE DÉVELOPPEMENT 3
LE SERVEUR WEB 3
CONCLUSION 5
LIENS INTERESSANTS 5

2. DÉROULEMENT DU COURS / EXAMEN 6

3. LES BASES DU HTML 8

PREMIERE PAGE HTML 8
LES BALISES 9
LES BALISES EN PAIRES 9
LES BALISES ORPHELINES 9
LES ATTRIBUTS 10
STRUCTURE DE BASE D'UNE PAGE HTML 11
LES PARAGRAPHES 13
LES TITRES 13
LES LISTES 14
LISTE NON ORDONNEE 14
LISTE ORDONNEE 14
LES LIENS 15
LES IMAGES 16
EN CONCLUSIONS 16
LIENS INTERESSANTS 16
EXERCICES 17

4. STYLE ET SÉMANTIQUE 18

LES DEUX FAMILLES DE BALISE 18
BALISE DE TYPE INLINE 18
BALISE DE TYPE BLOCK 18
SÉMANTIQUE 20
QU'EST-CE LA SÉMANTIQUE ? 20
POURQUOI DE LA SÉMANTIQUE ? 20
DU STYLE 22
LA FEUILLE DE STYLE 22
LES CLASSES CSS 23
LES IDS 23
HIÉRARCHIE DE STYLE 23

RESET DE STYLE 25
LA MISE EN PAGE / STRUCTURATION D'INTERFACE 26
FLEX 26
GRID 26
LIENS INTERESSANT 27
EXERCICES 27

5. CONTENU ENRICHI 28

LES FORMULAIRES 28
BALISE FORM 28
BALISE INPUT 28
BALISE TEXTAREA 32
CHECKBOX / RADIO 32
LISTE DÉROULANTE 33
VIDÉO / AUDIO 34
AJOUTER UNE VIDÉO 34
AJOUTER DE L'AUDIO 34
RESPONSIVE DESIGN 35
COMMENT ? 35
CONSTRUCTION DE CONDITION 36
QUELQUES EXEMPLES : 37
MOBILE FIRST OU DESKTOP FIRST 37
LES ANIMATIONS 43
EXERCICES 44

6. PROJET 45

SUJET 45
EXIGENCE TECHNIQUE 45
EXIGENCE QUANTITATIVE 45
DEROULEMENT DE L'EXAMEN 45

1. OUTILS

INTRODUCTION

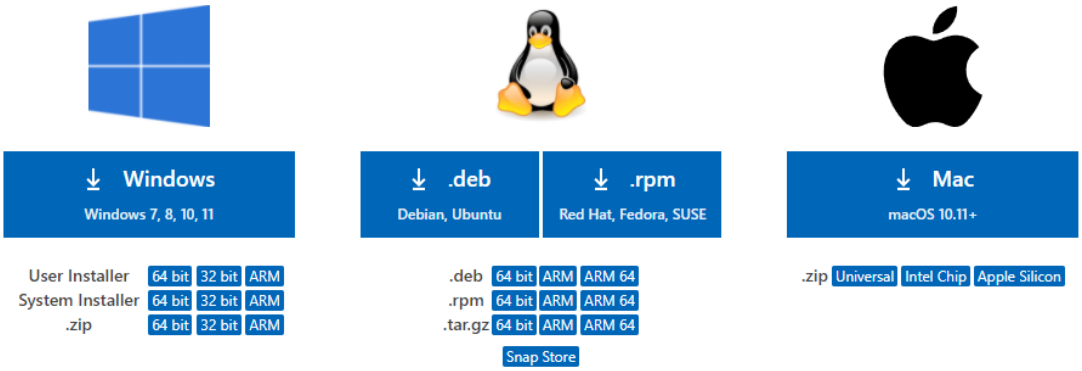
Dans le cadre de cette UE, l'étudiant sera amené à pratiquer le codage html et css. Pour ce faire, il devra utiliser des outils de développement (IDE) tel que Visual Studio Code, Sublime Text ou bien bloc note (non conseillé). La finalité étant la mise en place d'un projet web sur un serveur local, l'étudiant devra installer Laragon. La suite de ce chapitre est consacrée à l'installation et les premiers pas dans ces différents outils.

INSTALLATION DES OUTILS

L'ENVIRONNEMENT DE DÉVELOPPEMENT

Pour développer des interfaces web, le bloc note suffirait.
Cependant pour une question de confort et de productivité, nous utiliserons Visual studio code qui est gratuit.

https://code.visualstudio.com/download

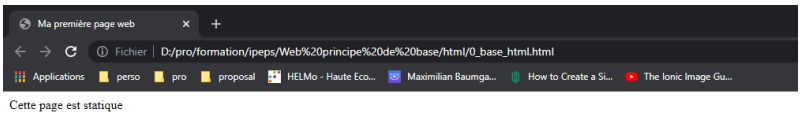


En fonction de votre système d'exploitation, choisissez l'installateur.

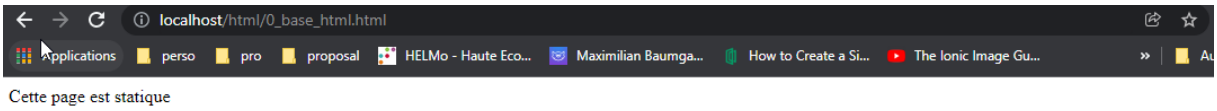
LE SERVEUR WEB

Pour pouvoir exposer vos pages html, vous devrez utiliser un serveur web.
Qu'est-ce un serveur web ? Selon Wikipédia, c'est un logiciel de service de ressources web (serveur http). Le protocole de communication HyperText Transfer Protocol (http) permet le dialogue via le réseau avec le logiciel client, généralement un navigateur web.

A savoir, qu'il est possible d'afficher une page web html statique via son chemin d'accès.



Voici un exemple via un serveur web



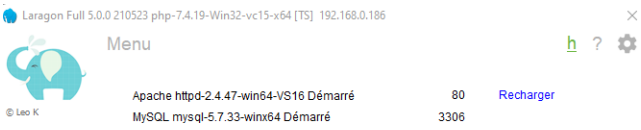
Comme vous pouvez le constater, il n'y aucune différence sur l'affichage d'une page html. Seul le lien change :

Chemin physique depuis votre poste	Lien HyperText
file:///D:/pro/formation/ipeps/Web%20principe%20de%20base /html/0_base_html.html	http://localhost/html/0_base_html.html

Quel serveur web allons-nous utiliser ? Nous utiliserons le serveur web Laragon

https://laragon.org/

Une fois installé, il vous sera proposé de lancer l'application



Apache	MySQL
Serveur web http qui nous permettra, au travers, de l'url de base « localhost » d'accéder à nos pages web.	Serveur de base de données, ceci ne fait pas l'objet du cours.

CONCLUSION

Ce qu'il faut retenir de ce chapitre :

1. Pour développer des pages html il vous faudra à minima un logiciel de traitement de texte tel que le bloc note mais il est conseillé d'utiliser un IDE tel que Visual studio code
2. Pour pouvoir exposer vos pages html à travers le web, il vous faudra un serveur web contenant à minima un serveur http

LIENS INTERESSANTS

https://fr.wikipedia.org/wiki/Apache_HTTP_Server
https://hackr.io/blog/web-development-ide
https://home.cern/fr/science/computing/birth-web/short-history-web
https://fr.wikipedia.org/wiki/World_Wide_Web

2. DÉROULEMENT DU COURS / EXAMEN

Le cours contient 40 périodes réparti en 10 séances :

Séance 1	Introduction
Séance 2	Base du html
Séance 3	Base du html
Séance 4	Styles et sémantiques
Séance 5	Styles et sémantiques
Séance 6	Contenu enrichi
Séance 7	Séance labo projet
Séance 8	Séance labo projet
Séance 9	Séance labo projet
Séance 10	Examen

Il est important de noter que ce planning est purement indicatif.

3. LES BASES DU HTML

PREMIERE PAGE HTML

```
erminal  Help
0_base_html.html   1_premiere_page_web.html ×
1_premiere_page_web.html
1   Ma premiere page web
2   Voici du contenu
3
4   et encore du contenu
```

Et voici le résultat

Fichier | D:/pro/formation/ipeps/Web%20principe%20de%20base/html/1_premiere_page_web.html

Applications perso pro proposal HELMo - Haute Eco... Maximilian Baumga... How to Create a Si... The Ionic Ir

Ma premiere page web Voici du contenu et encore du contenu

Quel constat pouvez-vous faire ?

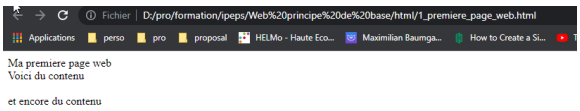
Nous avons créé un fichier avec l'extension .html dans lequel nous avons mis du texte.

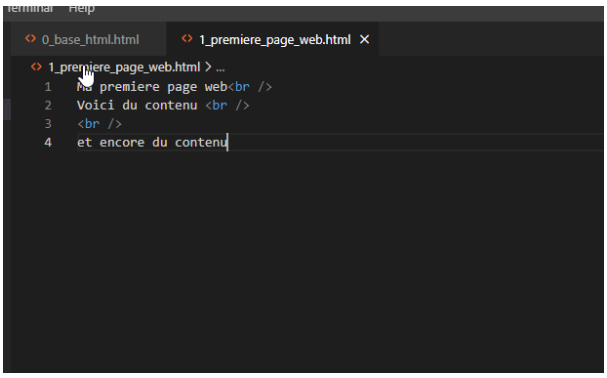
Ce texte apparait bien sur la page web mais pas dans le même format que celui écrit dans notre page !

La raison est simple ; pour pouvoir formater du texte, nous devons écrire du « code ».
Du code ?

C'est-à-dire un mélange de texte et de balises !

Voici ce que nous devrions faire pour formater les sauts de lignes





Nous allons voir tout cela dans la suite !

LES BALISES

Une balise dans le contexte web est un bout de code permettant de donner des instructions (principalement) d'affichage au navigateur. Ce bout de code est invisible pour les visiteurs.

Il existe deux grandes familles de balises :

- Les balises en paires
- Les balises orphelines

LES BALISES EN PAIRES

Cette famille de balise contient du « texte » ou d'autres balises voici comment l'écrire, elle contient donc une balise ouvrante et une balise fermante.

```
<balise-en-paire>

    Du contenu

    <balise-en-paire-2>Encore du contenu</balise-en-paire-2>

</balise-en-paire>
```

LES BALISES ORPHELINES

Cette famille de balise ne contient pas de contenu, elle sert globalement à insérer un élément tel qu'une image.

IL n'y a donc pas de balise ouvrant et fermante, mais une balise auto-fermante.

```
Du contenu

<balise-orpheline />

Encore du contenu
```

LES ATTRIBUTS

Les attributs permettent la configuration des balises. Ils se situeront au niveau de la balise ouvrante

```
<balise-en-paire attribut="mon attribut">

    Du contenu ici

    <balise-orphelin attribut="mon attribut également" />

    Encore du contenu

</balise-en-paire>
```

Exemple concret au travers d'une image :

```
Ma premiere page web<br />
Voici du contenu <br />
<br />
et encore du contenu<br />
<image src="ressources/images/photo.jpg" />
```

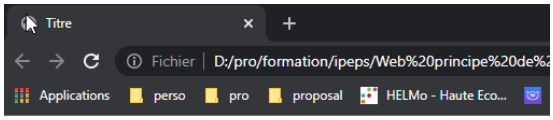
- De quel type de balise s'agit-il ?
- Pouvez-vous me citer l'attribut ?
- Pouvez-vous me donner le rôle de l'attribut ?

Nous verrons plus loin dans cette partie les principales balises d'une page web

STRUCTURE DE BASE D'UNE PAGE HTML

Avant de voir les principales balises html, nous devons structurer notre page html.

```html
<!DOCTYPE html>
<!-- Ceci est un commentaire-->
<html>
    <head>
        <!-- En-tête de la page -->
        <meta charset="utf-8" />
        <title>Titre</title>
    </head>

    <body>
        <!-- Corps de la page -->
        <p>
            Mon super contenu<br />
            Le br est une balise orpheline :)
        </p>
    </body>
</html>
```



Mon super contenu
Le br est une balise orpheline :)

Remarquez que l'ordre de fermeture des balises correspond à l'ordre d'ouverture

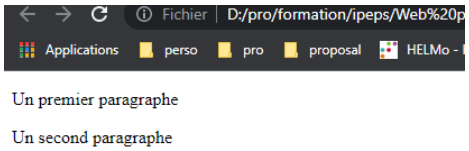
```html
<!DOCTYPE html>

<html>

    <head>

        <meta charset="utf-8" />

        <title></title>

    </head>



    <body>

        <p></p>

    </body>

</html>
```

Balise	Famille	Explication
Doctype	Orpheline	Permet d'indiquer qu'il s'agit d'une page web
Html	En paire	Balise principale du code, elle englobe tout le contenu
Head	En paire	L'en-tête est la carte d'identité de votre page web, elle contient l'encodage, le titre et d'autres informations telle que le lien vers le style.
Meta	?	C'est une balise permettant de définir des propriétés qui ne sont pas défini par d'autre balise tel que le title
Title	?	Balise permettant l'affichage du titre (nous verrons cela plus loin)
Body	?	Tout le contenu de votre page se situera entre les balises <body></body>
P	?	Balise permettant de créer un paragraphe
br	?	La balise de saut de ligne

LES PARAGRAPHES

Vous serez amené à structurer vos différents textes cela peut se faire grâce au paragraphe un peu comme dans un document word.
Vous pouvez le faire grâce à la balise p

```
<body>
        <p>Un premier paragraphe</p>
        <p>Un second paragraphe</p>
    </body>
```



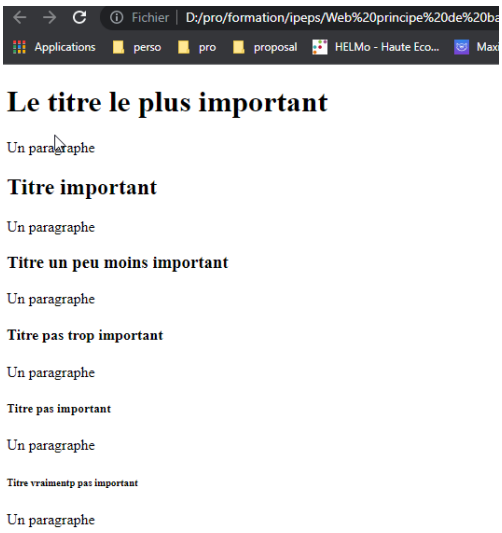
Remarquez que la balise fermante du paragraphe fait un saut à la ligne automatique ! Nous verrons cela dans la partie style de ce cours. Comme vous avez pu le constater, une page html ne comprend pas le saut de ligne « enter » que vous pourriez faire dans votre page web. Afin de faire un retour à la ligne, vous devez utiliser la balise orpheline

LES TITRES

Outre le titre dans la carte d'identité de la page web

```
<title>Mon super titre</title>
```

Comme un document texte (Word), nous avons la possibilité de « structurer » le contenu de notre page web grâce à différent niveau de titre




```
<h1>Le titre le plus important</h1>
        <p>Un paragraphe</p>
        <h2>Titre important</h2>
        <p>Un paragraphe</p>
        <h3>Titre un peu moins important</h3>
        <p>Un paragraphe</p>
        <h4>Titre pas trop important</h4>
        <p>Un paragraphe</p>
        <h5>Titre pas important</h5>
        <p>Un paragraphe</p>
        <h6>Titre vraiment  pas important</h6>
        <p>Un paragraphe</p>
```

LES LISTES

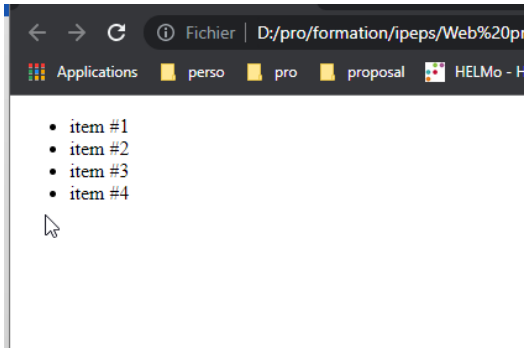
Encore une fois, comme dans un document texte, vous pouvez avoir besoin de liste. En html nous distinguons deux types de liste :

- Liste non ordonnée ou liste à puce
- Liste ordonnée ou liste numérotée

LISTE NON ORDONNEE

Afin d'écrire une liste non ordonnée il faudra utiliser la balise <ul></ul> et à l'intérieur de celle-ci nous aurons les éléments (<li></li>)

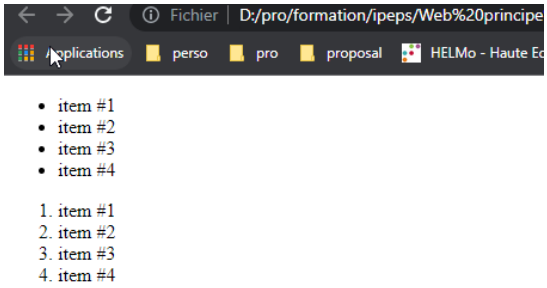
```
<ul>
            <li>item #1</li>
            <li>item #2</li>
            <li>item #3</li>
            <li>item #4</li>
        </ul>
```



LISTE ORDONNEE

La différence avec la liste non ordonnée est la balise parent <ul></ul> qui devient <ol></ol>

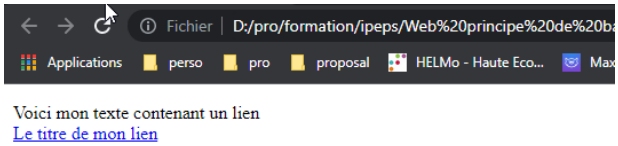
```
<ul>
            <li>item #1</li>
            <li>item #2</li>
            <li>item #3</li>
            <li>item #4</li>
        </ul>
        <ol>
            <li>item #1</li>
            <li>item #2</li>
            <li>item #3</li>
            <li>item #4</li>
        </ol>
```



LES LIENS

Jusqu'à maintenant, nous avons créés des pages html sans liens entre elle. Nous allons voir maintenant comment créer des liens vers d'autres pages internes ET d'autres pages externes

```
<p>
  Voici mon texte contenant un lien <br />
  <a href=http://www.google.com
target="_BLANK">Le titre de mon lien</a>
        </p>
```

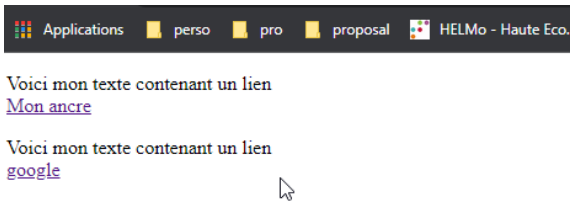


Balise	Attribut	Explication
a		La balise principale pour créer un lien
	href	L'attribut contenant l'url
	target	L'attribut target permettant de définir l'endroit où l'on va ouvrir le lien, dans le cas d'exemple nous voulons ouvrir le lien dans un nouvel onglet

Outre les liens vers d'autre page web – du même site ou d'un site externe – nous pouvons renvoyer vers une ancre.

Qu'est-ce qu'une ancre ? **Un point de repère que l'on met dans une page grâce à l'attribut id.**
Cela permet notamment de renvoyer à un endroit précis dans la page

```
<p>
        Voici mon texte contenant un lien <br />
        <a href="#monancre">Mon ancre</a>
        <!-- contenu du paragraphe-->
    </p>
    <p id="monancre">
        Voici mon texte contenant un lien <br />
        <a href="http://www.google.com" target="_BLANK">google</a>
        <!-- contenu du paragraphe-->
    </p>
```



LES IMAGES

Pour afficher une image, comme nous l'avons vu plus haut, il faudra utiliser la balise <img> qui est une balise orpheline :

```
<body>
  <img src="ressources/images/photo.jpg" alt="Ma super image">
</body>
```

Balise	Attribut	Explication
img		La balise principale pour afficher une image
	src	L'attribut contenant le chemin ou le lien vers l'image
	alt	Alt est un attribut obligatoire permettant d'afficher un texte alternatif

EN CONCLUSIONS

- Pour structure le code de votre page html vous utiliserez des balises pouvant contenir des attributs.
- Il existe deux grandes familles de balises (en paire et orpheline).
- Le langage Html fourni des balises pour créer des liens internes et externe au site mais aussi des ancres.
- Il existe une balise image.
- Il existe une série de titre h1 → h6

LIENS INTERESSANTS

https://fr.tuto.com/blog/2020/10/html.htm
https://www.w3.org/
https://developer.mozilla.org/fr/docs/Web/HTML/Element

EXERCICES

Voici une liste d'exercice vous permettant de prendre en main les notions vues ci-dessus

1. Créer une page web reprenant la structure de base nommé la : 0_structure_base.html
2. Créer une page web contenant le titre « ma première page »
3. Créer une page web contenant à minima trois paragraphes avec des sauts de lignes
4. Créer une page web contenant une bonne utilisation d'au moins 3 titres.
5. Créer une page web contenant un lien texte vers google.
6. Créer une page web contenant une image.
7. Créer une page web contenant un lien en image.
8. Créer une page web contenant une liste non ordonnée
9. Créer une page web contenant une liste ordonnée
10. Concevez un cv sans style reprenant les rubriques : introduction, parcours pro, qualité, motivation, contact.
11. Reprenez le cv (10) et enrichissez les différentes sections avec des images, des textes et des liens.

4. STYLE ET SÉMANTIQUE

LES DEUX FAMILLES DE BALISE

Maintenant que vous connaissez le concept de balise, nous allons voir les différents types qui existe :

- Balise de type inline
- Balise de type block

La plupart des balises html rentreront dans une des deux catégories.

BALISE DE TYPE INLINE

Une balise de type inline se trouve obligatoirement à l'intérieur d'une balise block.

Une balise inline ne crée pas de retour à la ligne, le texte qui se trouve à l'intérieur s'écrit donc à la suite du texte précédent, sur la même ligne (c'est pour cela que l'on parle de balise « en ligne »).

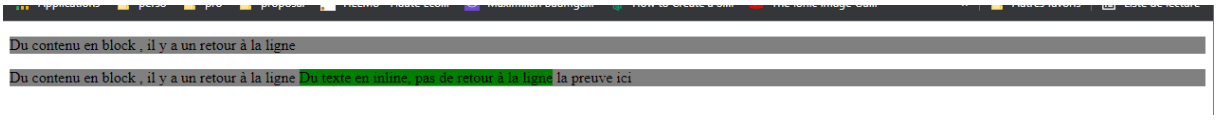
<a> , <abbr> , <acronym> , <area /> , <bdo> ,
 , <button> , <caption> , <cite> , <code> , <del> , <dfn> , , <img /> , <input /> , <ins> , <kbd> , <label> , <legend> , <link /> , <meta /> , <object> , <optgroup> , <option> , <param /> , <q> , <samp> , <select> , , , <sub> , <sup> , <textarea> , <title> , <var>

BALISE DE TYPE BLOCK

Une balise de type block sur votre page web crée automatiquement un retour à la ligne avant et après. Il suffit d'imaginer tout simplement un bloc. Votre page web sera en fait constituée d'une série de blocs les uns à la suite des autres. Mais vous verrez qu'en plus, il est possible de mettre un bloc à l'intérieur d'un autre, ce qui va augmenter considérablement nos possibilités pour créer le design de notre site !

<address> , <blockquote> , <body> , <dd> , , <dl> , <dt> , <fieldset> , <form> , <h1> , <h2> , <h3> , <h4> , <h5> , <h6> , <head> , <hr /> , <html> , <li> , <map> , <noscript> , <ol> , <p> , <pre> , <script> , <style> , <table> , <tbody> , <td> , <tfoot> , <th> , <thead> , <tr> , <ul>

Exemple,
Vous remarquez que la partie grise est une balise de type block, tandis que le partie verte est de type inline




```
<!DOCTYPE html>

<html>

    <head>

        <meta charset="utf-8" />

        <title>Mon super titre</title>

    </head>



    <body>

        <p style="background:gray">Du contenu en block , il y a un retour à la ligne</p>
```

```
        <p style="background:gray">Du contenu en block , il y a un retour à la ligne

            <span style="background:green">Du texte en inline, pas de retour à la ligne</span> la preuve ici</p>

    </body>

</html>
```

SÉMANTIQUE

QU'EST-CE LA SÉMANTIQUE ?

Si vous connaissez un peu le HTML, vous savez que les balises HTML sont principalement utilisées pour l'affichage de la page - ces balises indiquent au navigateur comment afficher le contenu de la page. Elles ne donnent aucune indication sur le type et le rôle du contenu présenté.

La sémantique comble cette lacune en définissant des balises spécifiques pour indiquer clairement le rôle du contenu qu'elles englobent. Cette information explicite aide les machines comme Google et Bing à comprendre les rôles et l'importance relative de différentes parties de votre page.

POURQUOI DE LA SÉMANTIQUE ?

La sémantique est util pour la bonne compréhension du code, et pour permettre au bot de google et autre (crawler) de comprendre la structure de votre page web.

Exemple

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Mon super titre</title>
    </head>

    <body>
        <header> mon header </header>
        <nav>
            <ul> <li>Lien du menu</li> </ul>
        </nav>
        <section>
            <aside> contenu secondaire </aside>
            <article>contenu principal</article>
        </section>
        <footer>mon super footer</footer>
    </body>
</html>
```

mon header

- Lien du menu

contenu secondaire
contenu principal
mon super footer

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Mon super titre</title>
    </head>

    <body>
        <div>mon header</div>
        <div>
            <ul> <li>Lien du menu</li></ul>
        </div>
        <div>
            <div> contenu secondaire</div>
            <div>contenu principal</div>
        </div>
        <div>mon super footer</div>
    </body>
</html>
```

mon header

- Lien du menu

contenu secondaire
contenu principal
mon super footer

DU STYLE

LA FEUILLE DE STYLE

Le style peut être inséré directement dans les balises html mais ce n'est pas conseillé.
La bonne pratique est donc de créer un fichier CSS.

Ce fichier CSS devra alors être lié à votre page html grâce à une balise insérée dans les balises Head

Quel est le rôle de la balise head ?

Quel est le rôle de la balise nav ?

Un exemple

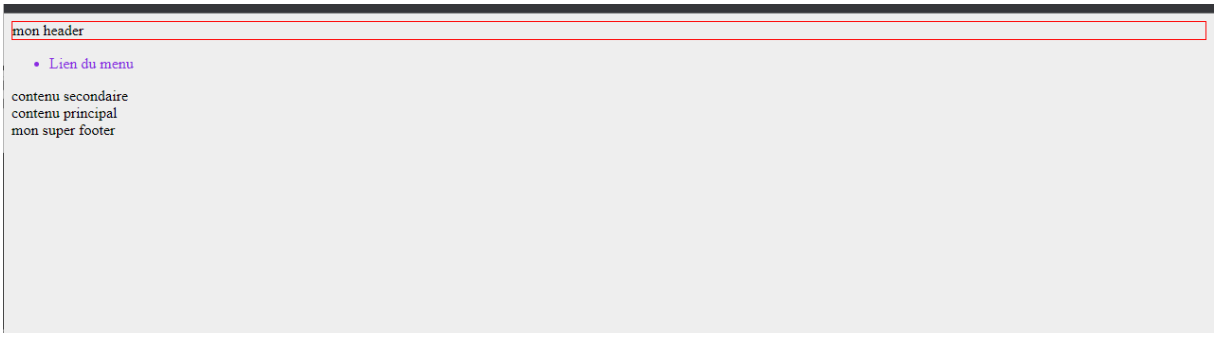
Le html

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Mon super titre</title>
        <link rel="stylesheet"
href="css/3_3_first_style.css" />
    </head>

    <body>
        <header> mon header </header>
        <nav>
            <ul> <li>Lien du menu</li> </ul>
        </nav>
        <section>
                <aside> contenu secondaire
</aside>
            <article>contenu principal</article>
        </section>
        <footer>mon super footer</footer>
    </body>
</html>
```

```css
body{
    background: #eeeeee;
}
header{
    border:1px solid red;
}
nav{
    color:blueviolet
}
```

Avant l'application de style

Après l'application de style



LES CLASSES CSS

Imaginez que vous vouliez un style unique pour tous les messages d'erreurs de votre application, Comment feriez-vous cela ?
La réponse est simple, il vous faudra utiliser une classe css qui regroupera les propriétés css.
Ensuite vous utiliserez cette classe dans l'attribut class de votre élément html

Exemple

```
<div class="color">
            couleur de texte
        </div>
```

```
.color{
    color:red;
}
```

LES IDS

Les ids vont exactement la même chose à la différence que :

- Il doit être unique dans la page html
- Le style est prioritaire (hiérarchie) à celui des classes css

HIÉRARCHIE DE STYLE

Il est bon de savoir qu'il existe un héritage naturel « parent-enfant » :

C'est-à-dire ? Body est un élément enfant de html. Cet héritage « naturel » ne fonctionne pas pour toutes les propriétés css, par exemple si je donne un padding à ma balise html, celle-ci ne sera pas transmise à la balise body. Cependant la taille du texte (font-size) le sera.

Outre l'héritage, il existe une importance dans l'ordre dans laquelle vous insérez vos classes css.

Quelques exemples, voici le div que nous allons utilisez ainsi que les différents styles.
Vous remarquez que des propriétés css sont overrider (réécrite plusieurs fois) dans les différentes classes. Lesquelles ?

```html
<div class="style1 style2 style3">
            style1 style2 style3
        </div>
        <div class="style1 style3 style2">
            style1 style3 style2
        </div>
        <div class="style3 style1 style2">
            style3 style1 style2
        </div>
        <div class="style3 style2 style1">
            style1 style2 style3
        </div>
        <div class="style2 style1 style3">
            style2 style1 style3
        </div>
        <div class="style2 style3 style1">
            style2 style3 style1
        </div>
```

```css
.style1{
    display: inline-block;
    width:200px;
    height:200px;
    border:8px solid yellow;
    color:white;
}
.style2{
    background:green;
}
.style3{
    background:red;
    color:violet;

}
```

Cas 1 :

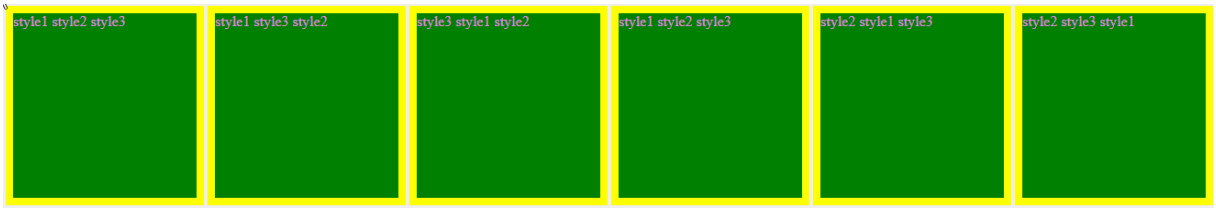
Dans la feuille de style nous définissons les style dans l'ordre suivant : 1,2,3 :

Que constatez-vous :

- Résultat ?
- Balises dans le code html ?
- Balises dans le code css ?

Cas 2

Dans la feuille de style nous définissons les style dans l'ordre suivant : 1,3,2 :



Que constatez-vous :

- Résultat ?

- Balises dans le code css ?

RESET DE STYLE

Pourquoi faire du reset de style ?

Listons les principaux navigateurs web :

- Firefox
- Opéra
- Chrome
- Edge
- Safari

Chacun de ces navigateurs possèdent une interprétation / design des balises html de base (input, p, etc…) Dans la globalité, ces balises html ont le même comportement, mais l'aspect varie. De ce fait, en resetant le style css , vous assurez une base commune à votre design quel que soit le navigateur choisi.
https://github.com/necolas/normalize.css

Afin de faire ce reset css, il vous suffira d'importer cette feuille de style

```html
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Mon super titre</title>
        <link rel="stylesheet" href="css/3_3_first_style.css" />
        <link rel="stylesheet" href="css/normalize.css" />
    </head>

    <body>
    </body>
</html>
```

LA MISE EN PAGE / STRUCTURATION D'INTERFACE

La mise en page ou structuration d'interface est une notion importante si vous voulez pouvoir créer de superbe page html , nous ne parlerons pas ici d'UX (User Expérience) cela pourrait faire l'objet d'un cours en particulier car c'est devenu une vrai science.
Nous nous contenterons de voir les bases de l'ux, en apprenant comment disposer nos différents éléments sur une page .

FLEX

Cette section de cours est fortement inspirée de ce site web (très bien fait)

https://www.devenir-webmaster.com/V2/TUTO/CHAPITRE/HTML-CSS/26-les-flexbox/#:~:text=Une%20Flexbox%20c'est%20une,flex%2Ditems%20dans%20ce%20dernier.

La mécanique Flex utilise principalement deux notions :
Le flex-container qui contiendra des éléments enfants appelés flex-items.

Afin de vous apprendre au mieux le flexbox, je vous propose de parcourir ce petit jeu en ligne :

https://flexboxfroggy.com/#fr
Les notions y sont bien expliquées !

GRID

Le grid css est une autre façon de concevoir des interfaces, il est notamment utilisé dans les framework tel que boostrap.
A nouveau, voici un petit jeu vous permettant de prendre en main la grille css
https://cssgridgarden.com/#fr

LIENS INTERESSANT

http://www.css-faciles.com/proprietes-css-liste-alphabetique.php

https://openclassrooms.com/fr/courses/1603881-apprenez-a-creer-votre-site-web-avec-html5-et-css3/1608902-memento-des-proprietes-css
https://developer.mozilla.org/fr/docs/Web/CSS/CSS_animated_properties

EXERCICES

1. Explorer les différentes balises inline block.
2. Créer une structure classique contenant :
    a. Un header contenant un titre du plus haut niveau
    b. Un élément de navigation contenant une liste non ordonnée et 5 liens
    c. Une section contenu du contenu principal et secondaire
    d. Un footer
3. Créer une page html liée à une feuille de style
    a. Créer une classe css pour une bordure de 3 pixels de couleur noir
    b. Créer une classe css pour une couleur d'écriture bleue
    c. Créer une classe css pour une marge intérieur de 10px
    d. Créer une classe css pour une marge extérieur de 20px
    e. Avec l'ensemble des classes ci-dessous faites les combinaisons suivantes :
    f. Un élément de type block avec une bordure et une marge intérieur ainsi qu'une largeur de 200 pixel
    g. Un élément de type inline avec une couleur d'écriture rouge
    h. Un élément de type inline avec une couleur d'écriture bleu
    i. Un élément de type block utilisant toutes les classes css crée au point a, b,c,d
    j. Dans la feuille de style, insérer un style générique pour toutes les balises « p » :
        i. Couleur jaune
        ii. Couleur de fond gris clair.
    k. Dans la feuille de style, insérer un style pour toutes les balises span d'un div dont l'id est mon-div :
        i. Couleur rouge
        ii. Text en gras
    l. Pour le point j et k, testez et démontrer le résultat.
    m. Utilisez le reset css

5. CONTENU ENRICHI

LES FORMULAIRES

Les formulaires permettent à vos visiteurs de vous transmettre des informations.
Nous allons prendre l'exemple d'un formulaire de contact, le but de ce formulaire serait de proposer à vos visiteurs de prendre contact avec vous.

BALISE FORM

Un formulaire sera toujours, d'un point de vue bonne pratique / sémantique, encapsuler dans une balise form.

```
<body>
        <h1>Prendre contact</h1>
        <form method="post" action="backend/traitement">
        </form>
    </body>
```

Cette balise prendra deux attributs :

1. Méthod
2. Action

Méthod

Cette balise permet de définir comment les données envoyées depuis le formulaire devront être récupérer. La récupération des données nécessite un langage plus évolué (java/ angular/ php) qui ne fait pas l'objet de ce cours. Dès lors retenez que dans la bonne pratique, nous utiliserons la méthode post pour envoyer des données.

Action

Défini l'url qui fera le traitement des données envoyées par le formulaire.

BALISE INPUT

La balise input est une zone de saisie monoligne, cela signifie que l'on ne pourra pas faire de retour à la ligne.

Voici un exemple que nous détaillerons dans le sous chapitre type text :

```
<body>
        <h1>Prendre contact</h1>
        <form method="post" action="backend/traitement">
            <label for="name">Nom</label>
            <input type="text" name="name" id="name" placeholder="Entrez votre
nom"
            required
            minlength="10" maxlength="50"/>
```

```
        <button type="submit">Envoyer</button>
    </form>
  </body>
```

Il existe tout une panoplie de champs pour agrémenter nos formulaires. Nous allons passer en revue la majorité d'entre eux

Type text

Ce type est le type de base, c'est un input contenant une zone de saisie :

```
<form method="post" action="backend/traitement">
  <p>
    <label for="name">Nom</label>
    <input type="text" name="name" id="name"
           placeholder="Entrez votre nom"
           required  minlength="10" maxlength="50"/>
  </p>
  <button type="submit">Envoyer</button>
 </form>
```

Prendre contact

Nom Entrez votre nom Envoyer

Analysons le code ci-dessus, vous remarquerez la présence de la balise form qui encapsule tout le formulaire. Ensuite vous trouverez une balise p (une balise de type block) pour la bonne sémantique.
Dans la balise P il y aura un label : le label est un élément html qui permet d'afficher un élément avant le champs de saisie. Si cet élément est correctement configuré, lors du clic sur celui-ci, vous serez automatiquement focus sur le champ de saisie associé.
Et finalement vous aurez la balise input

Attribut	Description
Type	Défini le type d'input
Name	Donne un nom à l'input
Id	Permet d'associer un label à un
Placeholder	Permet d'afficher un message lorsque l'input est vide
Required	Attribut de validation html5 définissant que l'input doit être rempli pour que le formulaire soit valide
Minlength maxlength	Permet de définir une taille minimum et maximum pour la chaine de caractère.

Type password

Ce type possède les mêmes attributs que le champs de type text. Cependant les données insérées sont remplacées par un rond(par exemple) afin de cacher les valeurs encodées.

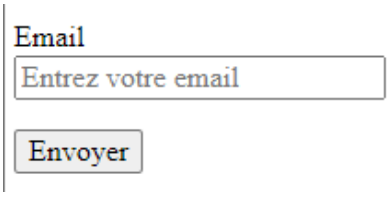
Attention que ceci cache la valeur dans le champ, mais n'est absolument pas une sécurité en soit.

```
<p>
   <label for="password">Mot de passe</label><br />
   <input type="password" name="password" id="password"
          placeholder="Entrez votre mot de passe"
          required  minlength="10" maxlength="50"/>
</p>
```

Type email

Outre le fait d'avoir les mêmes attributs que le champ text, le type email possède une validation html5 : la validation email.
A vous de tester les limites de cette validation.

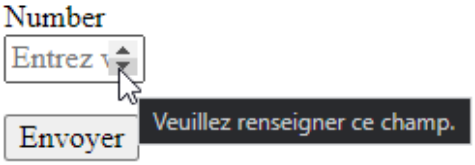
```
<p>
   <label for="email">Email</label><br />
   <input type="email" name="email" id="email"
           placeholder="Entrez votre email"
           required  minlength="10" maxlength="50"/>
</p>
```



Type number

Le type number permet de cadenasser la valeur à un numérique , plus précisément à un entier !!
Voici un exemple qui défini une limite haute et basse

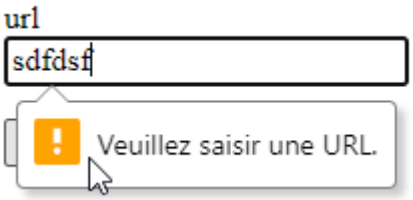
```
<p>
 <label for="number">Number</label><br />
 <input type="number" name="number" id="number"
        placeholder="Entrez votre nombre"
        required  min="10" max="50"/>
</p>
```



Type url

Ce type permet d'encoder une url , c'est surtout utile pour l'utilisation mobile dont le clavier virtuel (votre clavier sur le smartphone) propose alors une interface contenant un « http » et généralement un « .com »
De plus il y a une validation de type url , si le champs n'est pas considéré comme une url valide alors le formulaire est invalide

```
<p>
   <label for="url">url</label><br />
   <input type="url" name="url" id="url"
          placeholder="Entrez votre url"  required />
</p>
```



Type tel

Tout comme le type url (ainsi qu'email) , le champ de type tel permet surtout d'avoir un clavier adapté a la saisie d'un numéro de téléphone.
Malheureusement, ce champ ne possède aucune vérification html.

Type curseur

Le type curseur change complètement le design de l'input, ce type permet de proposer un curseur variant d'une valeur min jusqu'à une valeur maximum que vous pouvez définir.

```
<p>
  <label for="range">range</label><br />
  <input type="range" name="range" id="range"
placeholder="Entrez votre valeur" min="10" max="70"
                        required />
            </p>
```

Type color

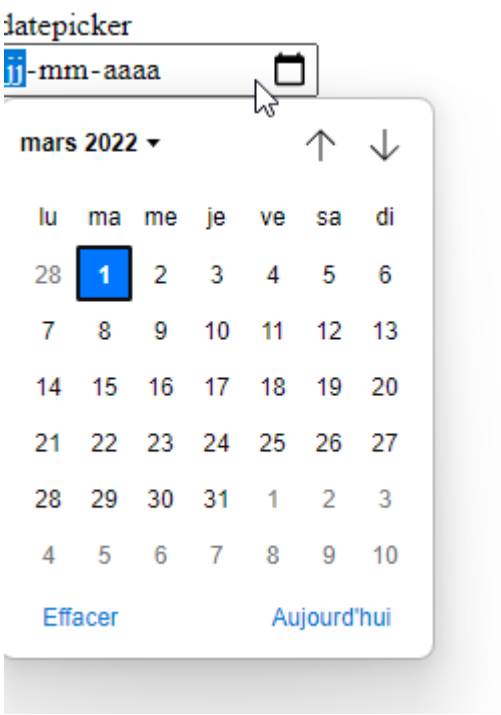
Le type color vous propose un color picker

```
<p>
 <label for="color">color</label><br />
 <input type="color" name="color" id="color"
        placeholder="Entrez votre color" required />
</p>
```

Type datepicker

Le type date permet de fournir un calendrier pour faciliter la selection d'une date.

```
<p>
<label for="datepicker">datepicker</label><br />
<input type="date" name="datepicker" id="datepicker"
       placeholder="Entrez votre date" required />
</p>
```

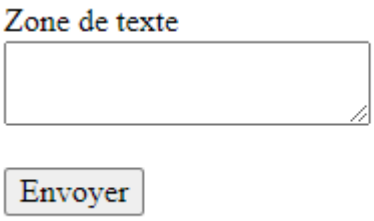


Ceci clôture l'introduction sur la partie champ de saisie mono ligne.
Pour aller plus loin je vous invite à parcourir la documentation :
https://developer.mozilla.org/fr/docs/Web/HTML/Element/Input

BALISE TEXTAREA

La textarea est une zone de saisie multi ligne.

```
<p>
<label for="textarea">Zone de texte</label><br />
<textarea name="textarea" id="textarea"
         placeholder="Zone de texte" required >
</textarea>
</p>
```



Voici la documentation officiel sur ce type de champ.
https://developer.mozilla.org/fr/docs/Web/HTML/Element/Textarea

CHECKBOX / RADIO

Les formulaires html permettent également un type d'input permettant de choisir des valeurs prédéfinies.

Les checkbox et les radio (il existe aussi les select nous verrons cela dans le prochain sous chapitre)

```html
<p>
 <input type="checkbox" id="check1" name="check1" checked>
 <label for="check1">check1</label>
</p>
<p>
 <input type="checkbox" id="check2" name="check2">
 <label for="check2">check2</label>
</p>
```

check1

check2

Envoyer

```html
 <p>
   <input type="radio" id="radio1" name="radio1" >
   <label for="radio1">radio1</label>
</p>
<p>
 <input type="radio" id="radio2" name="radio1">
 <label for="radio2">radio2</label>
</p>
<button type="submit">Envoyer</button>
```

radio1

radio2

Envoyer

La différence entre les deux hormis le design, c'est le fait de pouvoir sélectionner plusieurs valeurs.
Dans le cas des checkbox , vous pouvez choisir plusieurs valeurs.
Dans le cas des radios, vous pouvez, si vous le souhaitez, n'en choisir qu'une.

LISTE DÉROULANTE

Les listes déroulantes, comme pour les checkbox et radio, permettent la sélection d'une valeur prédéfinie.

```html
 <p>
   <label for="select">Select</label>
   <select id="select">
     <option value="1">1</option>
     <option value="2">2</option>
     <option value="3">3</option>
     <option value="4">4</option>
   </select>
 </p>
```

Select 2

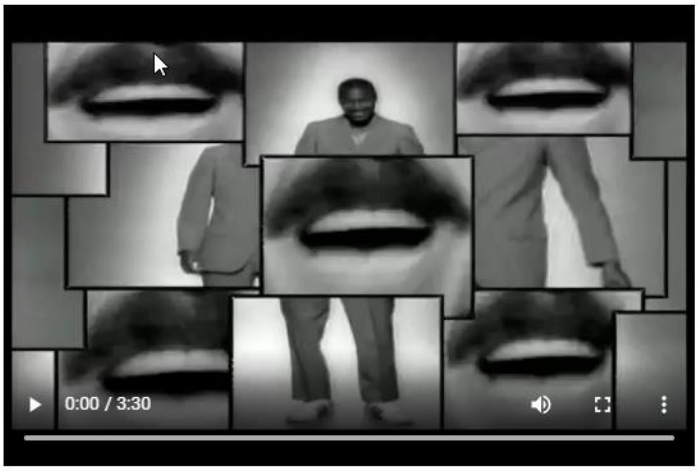
Envoyer

VIDÉO / AUDIO

AJOUTER UNE VIDÉO

Vous pouvez ajouter une vidéo grâce à cette balise

```
<p>
 <video src="ressources/video.mp4"
        controls></video>
 </p>
```

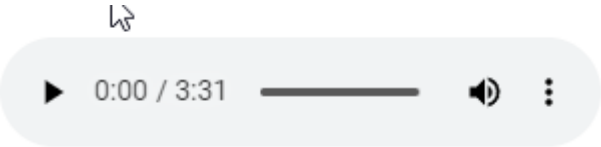


Attribut	Description
Poster	Permet de mettre une miniature
Controls	Permet d'ajouter les controls classique pour une vidéo
Width / height	Permet de définir la largeur et la hauteur de votre video
Loop	Défini si la vidéo est jouée en boucle
Autoplay	Défini s'il faut lancer la vidéo automatiquement
Preload	Permet de définir si le navigateur doit précharger la vidéo

AJOUTER DE L'AUDIO

De la même manière, il est possible d'ajouter une piste audio

```
<p>
 <audio src="ressources/musique.mp3"
        controls></audio>
</p>
```



RESPONSIVE DESIGN

Maintenant que vous savez créer et structurez du contenu enrichi. Nous allons voir comment gérez les différents format de vos écrans.

En effet, le rendu sur votre smartphone ne sera pas identique à celui de votre tablette ou même de votre pc. Il existe même différent format d'écran !
La meilleure façon de gérer cela est le responsive design.

Derrière ce terme se cache simplement une technique qui vous permet de définir des règles en fonction du format de votre terminal.

COMMENT ?

Il y a deux façons de faire :

1. Directement dans l'import du fichier
2. Dans le fichier css directement

Directement dans l'import du fichier

Comme vous avez pu déjà le pratiquer, vous avez importer votre style depuis le head de votre page html :

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Mon super titre</title>
        <link rel="stylesheet" href="css/normalize.css" />
        <link rel="stylesheet" href="css/4_6_responsive.css" />
    </head>

    <body>
    </body>
</html>
```

Imaginons que vous voulez importer le fichier 4_6_small_resolution.css seulement dans le cas où nous sommes sur un écran et une taille maximale à 1280px

```
 <link rel="stylesheet" media="screen and (max-width: 1280px)"
      href="4_6_small_resolution.css" />
```

Directement dans la feuille de style

Si vous préférez, vous pouvez mettre directement la condition dans le fichier de style

```
@media screen and (max-width: 1280px)
{
```

```
}
```

CONSTRUCTION DE CONDITION

Vous pouvez pratiquement créer toutes les combinaisons possibles.

Pour ce faire voici la liste des conditions :

Condition	Effet
Height	Hauteur de la zone d'affichage ; min-height, max-height , height
Width	Largeur de la zone d'affichage ; min-width, max-width , width
Device-height	Hauteur de périphérique ; min, max,
Device-width	Largeur du périphérique ; min, max
Orientation	Orientation du périphérique (portrait, paysage)
media	Type d'écran : 1. screen : écran classique 2. Handled : périphérique mobile 3. print : impression 4. tv : télévision 5. projection : projecteur 6. all : tous les types

Vous pouvez ensuite (comme pour des conditions en c ou c#) combiné des conditions :

Condition	Description
only	Uniquement
and	et
not	non

QUELQUES EXEMPLES :

Voici quelques exemples (exemple repris de l'excellent cours openclassroom)

```
/* Sur les écrans, quand la largeur de la fenêtre fait au maximum 1280px */

@media screen and (max-width: 1280px)

/* Sur tous types d'écran, quand la largeur de la fenêtre est comprise entre
1024px et 1280px */

@media all and (min-width: 1024px) and (max-width: 1280px)

/* Sur les téléviseurs */

@media tv

/* Sur tous types d'écrans orientés verticalement */

@media all and (orientation: portrait)
```

MOBILE FIRST OU DESKTOP FIRST

Qu'est-ce que le mobile ou desktop first ?

Les deux sont des approches de la conception de vos feuilles de style.

Cela va dépendre simplement de savoir si votre site web devra être visible ou s'il sera majoritairement visité sur mobile ? Si la réponse est oui alors il est conseillé de faire du mobile first, si la réponse est non alors du ce sera du desktop first.

Les bonnes pratiques actuelles laissent à penser que le mobile first est la meilleure approche.

Voici un super lien qui explique cela :

https://zellwk.com/blog/how-to-write-mobile-first-css/#:~:text=A%20mobile%2Dfirst%20approach%20to,uses%20min%2Dwidth%20media%20queries.&text=In%20the%20example%20above%2C%20%3Cbody,a%20red%20background%20below%20600px.

Exemple concret du responsive en utilisant le flexbox :

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Mon super titre</title>
        <link rel="stylesheet" href="css/normalize.css" />
        <link rel="stylesheet" href="css/4_6_responsive.css" />
    </head>

    <body>
        <header>Mon header</header>
        <section id="content">
        <nav>
            <ul>
                <li><a href="">Menu item #1</a></li>
                <li><a href="">Menu item #2</a></li>
                <li><a href="">Menu item #3</a></li>
            </ul>
        </nav>
        <section>
            <article>
                <h1>Mon article</h1>
                <p>
                    Lorem ipsum dolor sit amet consectetur adipisicing elit.
Tenetur nulla, quis accusantium perferendis explicabo qui rem esse ducimus
iure corrupti excepturi molestias in reprehenderit deserunt, dolore porro!
Sequi, vero ex.
                </p>
                <p>
                    <video src="ressources/video.mp4" controls></video>
                    <audio src="ressources/musique.mp3" controls></audio>
```

```
                </p>
            </article>
            <aside>
                <h2>aside part</h2>
                <ul>
                    <li>li #1</li>
                    <li>li #2</li>
                    <li>li #3</li>
                </ul>
            </aside>
        </section>
        </section>
    </body>
</html>
```

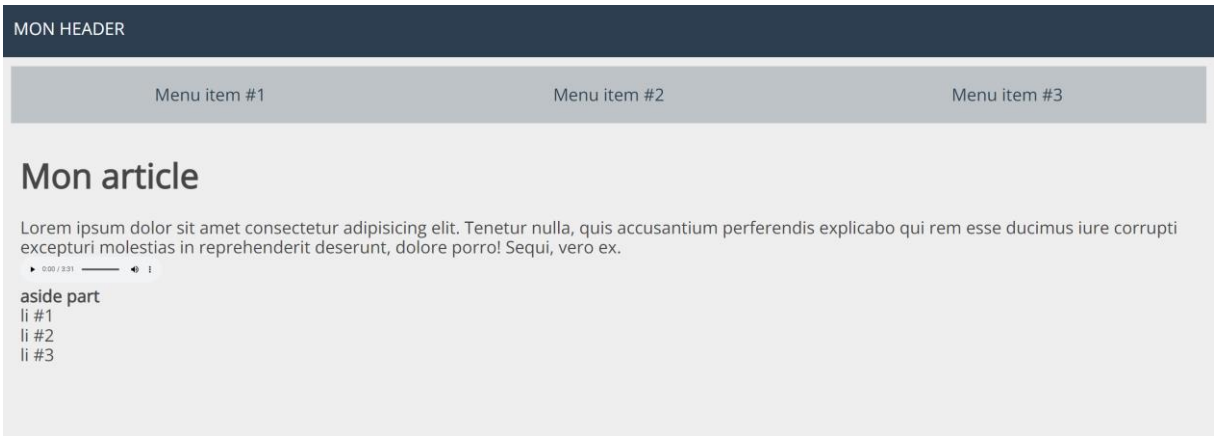
Voici le css orienté mobile first :

```
*{
    padding:0;margin:0;
    font-size:1.5rem; font-family: 'Open Sans', sans-serif;
}
body,html{
    background:#eeeeee; color:#464646;
}
/* Je ne veux pas afficher des vidéos lorsque je suis sur mobile*/
video{
    display: none;
}
/* Je défini ici que tous les éléments de type block doivent
    organiser leurs éléments enfants de manière verticale */
section, aside, article{
    display: flex;    flex-direction: column;
```

```
}

/* je retires les puces */

li{

    list-style: none;

}

a, a:visited{

    text-decoration: none;    color:#2c3e50;

}

/* Pour les ul dans les navs */

nav ul{

    display: flex; flex-direction: row; flex:1;

    height:100%;

}

nav ul li{

    flex:1;display: flex; flex-direction: column; justify-content: center;align-items: center;

    cursor:pointer;

}

/* Design du header */

header{

    height:120px; background:#2c3e50; color:#ffffff; font-size:1.5rem;

    display: flex; flex-direction: row;justify-content: center; align-items: center; text-transform:uppercase;

}

/* Design du nav */

nav{

    height:120px; background:#bdc3c7; color:#2c3e50

}

section{

    padding:0.8rem;

}
```

Et voici l'affichage version mobile :

Et version desktop



Nous allons maintenant modifier notre css pour que l'affiche desktop (web) change un peu :

```
@media screen and (min-width:1024px) {
    body,html{
        font-size:16px;
    }
    header{
        padding-left:1rem; justify-content: flex-start;
    }
    section{
        padding:0;
    }
    #content{
        display: flex; flex-direction:row; flex:1;
    }
    nav{
        flex:1; flex-direction: column; justify-content: flex-start; align-
items: flex-start; min-height:100vh; min-width:250px;
    }
    nav ul{
        display: flex; flex-direction: column; justify-content: flex-start;
align-items: flex-start; flex:1;
    }
    nav ul li{
        flex:0; padding:10px; width:100%;
    }
    nav ul li:hover{
        background:#eeeeee;
    }
    .p-10{
        padding:10px;
    }
}
```

Et voici le résultat final :

LES ANIMATIONS

Pour aller plus loin, vous pouvez appliquer les animations

https://developer.mozilla.org/fr/docs/Web/CSS/animation

EXERCICES

4. Créer un formulaire de type post dont la page de traitement est « backend/mon-traitement »
5. Créer un formulaire de type get dont la page de traitement est « backend/get-traitement »
6. Créer un formulaire contenant un champ texte requis et limité à 25 caractères
7. Créer un formulaire contenant un champ texte requis contenant entre 15 et 30 caractères
8. Créer un formulaire contenant un champ password requis et limité à 25 caractères
9. Créer un formulaire contenant un champ number minimum 10
10. Créer un formulaire contenant un champ number maximum 15
11. Créer un formulaire implémentant tous les champs
12. Implémenter un formulaire permettant de sélectionner plusieurs UE (ue1,ue2, ue3, ….)
13. Implémenter le même formulaire permettant de sélectionner qu'une seule ue

6. PROJET

SUJET

L'étudiant choisira un sujet qui pourra couvrir les exigences techniques et quantitatives.

Cependant le projet devra rester dans le cadre légal (pas de sujet illégal).

EXIGENCE TECHNIQUE

Le projet devra démontrer les acquis de l'étudiant.
Cela se fera au travers de la technique d'écriture des pages html ainsi que le respect de la sémantique.
Le site web devra avoir un design soigné et des feuilles de styles respectant les normes.
Le site web devra être responsive sur au moins les différents formats :

- Mobile
- Petit écran (en dessous d'une largeur 1024px)
- Ecran moyen (en dessous d'une largeur de 1600px)
- Et au dela .

Le site web sera exploitable au travers d'un réseau local.
La navigation se fera au travers d'un menu

EXIGENCE QUANTITATIVE

Le site web contiendra :

- Une page d'accueil
- Une page contact (reprenant un formulaire avec de multiple champs)
- Deux pages exploitant du contenant enrichi

Il faudra donc à minima quatre pages.

DEROULEMENT DE L'EXAMEN

L'étudiant exposera son projet devant l'entièreté de la classe lors de la dernière séance de cours.
Vous trouverez ci-dessous la grille d'évaluation.